



Опрос про ИИ внутри разработки

[Войти](#)

ushliypakostnik

40 минут назад

RUINSWORLD: Возвращение невозможной игры-предупреждения

 Средний 12 мин 1.1K

Браузеры, Веб-разработка*, TypeScript*, Дизайн игр*, Киберпанк

Все эти годы пафосные программисты говорили мне что это совершенно точно невозможно осуществить на только ноде, в браузере и на ts. А я сделаю.

Невозможная игра-предостережение некоторое время назад была все-таки дописана в первой играбельной минимальной версии, и вот можно уже тыкать и умирать <https://ruinsworld.online/>. Не забывайте пить воду, есть, охотиться. От ранений, отравлений и жажды начинает сильно качать. У вас есть союзники — это предки Роботов-Собутыльников... На самом деле это все рюшечки, а технически решение интересно тем, что это реально эффективно работающая реализация шовного открытого мира и шутера в нем, в бро, только исключительно на ноде и ts. Писал пять лет ручками, такое уже не модно нынче. Мир сейчас развернут на всего 9 локаций (и это число тоже может быть 25 вполне), но одной переменной его можно развернуть в четыре или девять, двадцать пять копий-процессов — зависит только от мощностей. Вот у меня одно ядро и поэтому только один процесс, один кластер-биом мира. Больше копий сервера — образуют замкнутую на себя сетку практически никак не связанных «частей мира», между которыми клиенты несущие много своего состояния могут свободно перемещаться. Нагрузка практически не прирастает, только немного при подключении новых игроков. Ну вот это и есть, что мы, по сути, построили стабильную модель открытого мира любой величины.

Это вторая и, видимо, последняя статья о многопользовательском браузерном шутере на TypeScript. Сказать, что мне надоело, — это не сказать ничего. Я просто адово упоролся. Но ни о чем не жалею.



Самое главное, о чём нужно рассказать в заключительном тексте об этой нелепой разработке, — это, скажем так — идеология «мультисервера». Осознав реальные ограничения среды и инструментов, которые были сформулированы в ходе долгой работы над проектом (смотрите конец [первой статьи](#)), я подумал, и, кажется, реализовал самый простой эффективный способ, получить из уже имеющегося у меня работающего решения на N в квадрате локаций, действительно большой открытый шовный мир — такой, что его размеры зависят только от мощности имеющегося на сервере железа. Получившаяся архитектура максимально гибкая, несвязная, и, наверное, является хорошим примером событийно-ориентированной архитектуры. Решение, в котором одной переменной можем запускать любое количество серверов — копий одного и того же процесса, — я реализовал с помощью менеджер процессов PM2.



Вместо одного сервера на всю игру — сетка серверов, каждый обслуживает свою часть карты. Это позволяет детерминировать и легко масштабировать нагрузку. Wrap-around (тороидальная топология) — при выходе за границу карты игрок переходит на противоположную сторону — мир «заворачивается» в тор. Каждый сервер может разворачиваться с определенным «скином» — биомом. Биомы могут получать разные настройки собственной генерации по конфигу, или даже наборы ассетов уже на клиенте.

MULTIWORLD	Сетка	Серверов	Порты
-----	-----	-----	-----
1	1×1	1	3001
2	2×2	4	3001-3004
3	3×3	9	3001-3009
4	4×4	16	3001-3016
5	5×5	25	3001-3025

Объяснить с  SourceCraft

Формула порта:

```
BASE_PORT = 3001
port = 3001 + y * width + x
```

Объяснить с 

MULTIWORLD=2 (2×2 сетка)

y=1 (север)

0_1	1_1
forest	town
0_0	1_0
town	desert

x=0 x=1
(запад) (восток)

SERVER	Координаты (x,y)	Порт	Биом
-----	-----	----	-----
0_0	(0, 0)	3001	town
1_0	(1, 0)	3002	desert
0_1	(0, 1)	3003	forest
1_1	(1, 1)	3004	town

Генерация серверов

▸ [Код](#)

Запуск: PM2 + ecosystem.config.js

Для запуска всех серверов используется PM2 с динамической конфигурацией:

▸ [Код](#)

1. Игрок выходит за границу локации (x = -1 при WORLD=1)
2. Клиент вычисляет соседний сервер через /config API
3. Клиент переключает socket на новый порт
4. Клиент отправляет событие релокации старому серверу
5. Клиент делает `window.location.reload()`
6. Новый сервер принимает игрока на новой локации

И вот тут можно рассказать о интересном. Можете себе представить, как мне было непросто тестировать переходы? Если каждый сервер у нас развёрнут на 9 локаций, то для того, чтобы добежать до соседнего сервера, нужно пройти полторы большие

локации. Тестировать такой переход приходилось очень много раз — и вот тут действительно выручает то, что всё решение действительно дотошно гибко и аккуратно написано: оно полностью опирается на базовые числа, и одной переменной можно изменить размер локации, количество локаций в сервере, отключить врагов, чтобы проще было бежать к горизонту и мы не получали урон... Ну, вот так вот я и выкручивался.

Серверы практически ничего не знают друг о друге. Единственное, что объединяет их — общий конфиг.

```
// /config API возвращает:
{
  "servers": [
    {
      "id": "0_0",
      "port": 3001,
      "view": "town",
      "top": "0_1",
      "bottom": "0_0",
      "left": "1_0",
      "right": "1_0"
    },
    // ... другие серверы
  ]
}
```

Объяснить с 

Каждый сервер знает только:

- Свой ID и порт
- Какой биом (town, forest, desert)
- ID соседей (top, bottom, left, right)

Серверы не общаются между собой напрямую.

Полный цикл релокации (между серверами)

1. КЛИЕНТ → доходит до края карты
2. КЛИЕНТ → вычисляет направление (top/bottom/left/right)
3. КЛИЕНТ → смотрит в конфиг: `server[direction]` → ID целевого сервера
4. КЛИЕНТ → вычисляет новые координаты (`newX`, `newY`) на основе направления
5. КЛИЕНТ → СЕРВЕР → отправляет: `relocationServer { id, location, newX, newY }`
6. СЕРВЕР → удаляет игрока (НЕ отправляет ответ!)
7. КЛИЕНТ → `setServer(newPort)` – переключает socket на новый сервер
8. КЛИЕНТ → `emitter.emit(EmitterEvents.relocationServer)` – отправляет
9. КЛИЕНТ → `window.location.reload()` – перезагрузка
10. НОВЫЙ СЕРВЕР → получает `onConnect` с `message.relocation = { user, newX, newY }`
11. НОВЫЙ СЕРВЕР → вычисляет `newLocationId = getLocationIdByCoords(newX, newY)`
12. НОВЫЙ СЕРВЕР → добавляет игрока на локацию, отправляет `onEnter`

Объяснить с 

При переходе между серверами координаты инвертируются для того чтобы сохранить «направление» движения.

► [Код на сервере](#)

► [Код на клиенте](#)

Перезагрузка — это нормально

И тут мы опять возвращаемся к теме, которая важной смысловой линией проходит через все мои статьи об веб-играх — кажется она критически важна для понимания правильной работы хорошего браузерного интерфейса, и, в том или ином виде, встречается повсеместно на клиентской стороне, в современных хороших практиках работы с поведением веб-страниц — вы должны думать о перезагрузке, прежде всего. Нам очень часто нужно сохранять и восстанавливать что-нибудь — то есть пробрасывать состояние через перезагрузку — умение адекватно обрабатывать такие сценарии критически важно. В данном случае, игра должна начаться, условно, «с того же самого места». Хотя если перезагрузка произошла случайно, вполне вероятно, что за это время игрок может получить урон от противников и погибнуть.

Но даже больше — RUINSWORLD это браузерный рогалик в котором перезагрузка страницы является именно штатным механизмом работы. Вкладка может быть закрыта/перезагружена в любой момент пользователем или системой (нехватка памяти, сон устройства). Сеть может быть нестабильна. Или разные биомы могут содержать разные ассеты (текстуры, 3D-модели, звуки) — намного проще загрузить новые, чем управлять переключением (сомневаюсь что вообще возможно реализовать корректно в бро с Three). Меньше багов, проще отладка, чище код — нет необходимости синхронизировать сложное состояние между локациями. Это НЕ баг — это фича! Не нужно бороться с браузерной средой — нужно принять её.

1. Игрок переходит на новую локацию (доходит до границы)
2. Клиент получает onRelocation с location.id
3. Клиент вызывает getLocation(location.id) → загружает данные локации

4. Срабатывает `watch(locationData) → world.init(self)`

5. Мир полностью пересоздаётся с новыми ассетами

6. Игрок появляется на новой локации

Сценарии перезагрузки:

- Переход между серверами: игрок дошёл до границы сервера → reload → подключение к новому серверу
- Переход между локациями: игрок дошёл до границы локации ($x = \pm \text{WORLD}$) → внутренняя релокация (wrap-around)
- Разрыв соединения: сеть оборвалась → перезагрузка → переподключение
- Заккрытие вкладки: игрок случайно закрыл → переоткрытие → вход заново

Расовая война на руинах: геноциды и эпидемии

В первой статье я рассказывал об оптимизационных механиках, которые ввел для того, чтобы улучшить производительность своей открытой веб-вселенной. Например, NPC на сервере в локациях, на которых нет игроков — замедляются, время там течет в 10 раз медленнее, и алгоритм динамически выбирает из массива NPC небольшое количество, которое необходимо в этом кадре немного «продвинуть». Потому что если они будут «совсем замирать», это крайне негативно скажется на игровом процессе. Они всё-таки должны медленно продвигаться по своей непростой жизни. Так вот, очень интересно: для хорошего баланса параметров различных игровых рас мне пришлось придумать еще

и механику эпидемии. Параметры игровых рас отвечают за совершенно разные вещи — например, интеллект у меня отвечает за то, насколько велик угол обзора у данного вида неигровых персонажей. И сбалансировать это очень сложно — всё равно какая-нибудь раса начинает доминировать и геноцидировать неудачников. И тогда я придумал эпидемии. Это ленивая проверка на сервере, которая очень редко пересчитывает количество особей данной расы, и если их становится слишком много — просто косит самых старых и опытных. В игре у каждого юнита с ходом временем увеличивается опыт, который влияет на его боевые качества (циферка над головой).

Слишком быстрые пули

Или вот ещё полезное. Я столкнулся со стандартной проблемой в шутерах. Быстрые пули проскакивали сквозь врагов. Обычный кастинг, даже «с подшагами» — не гарантировал обнаружение. И вот вам решение. Вместо точечного raycast — для надежной фиксации попаданий быстролетающих выстрелов нужно использовать swept-sphere (капсула от prevPos до currPos с радиусом снаряда). Проверяется вся траектория снаряда за кадр, а не только конечная точка.

► [Bullets.ts](#)

► [Octree.ts](#)

Для гранат используется пятикратно увеличенный радиус для учёта параболической траектории. И первые пять метров от точки вылета — зона безопасности (не проверяем мир и двери, чтобы не взрываться об стены рядом с игроком).

Нестандартные подходы для нестандартных задач!

Какой вывод можно сделать из всей этой истории с написанием странной игры в которую никто, вероятно, даже не будет играть? Невозможное — возможно. Главное — очень хотеть и стараться. При решении нестандартных задач эффективными оказываются именно нестандартные подходы. В моих реализациях, в принципе, нет ничего прямо революционного — просто некоторые «общепринятые» вещи, которые мы уже привыкли воспринимать определённым образом, были немного отодвинуты в сторону, и заменены вполне достаточными в данном случае по геймплею простыми решениями. Об этом много рассказывалось в первой статье. Например, то, что клиент сам считает свои выстрелы, позволяет очень мощно разгрузить сервер. И после того, как была решена проблема с пропуском попаданий — игровой процесс при этом остаётся достаточно динамичным, минимальная пиф-паф боевка не страдает от того, что у нас нет классического для шутеров тяжелого сервера как единственного источника истины, который делает и хранит слепки состояния и так далее...

Конечно, сейчас в игре делать практически нечего. Можно, прикрываясь сильными союзниками, пытаться валить опытных врагов и лутать их трупы — за счёт этого круто качаться. Ну, или шариться и щемиться на стелсе пытаюсь найти и заслушать потёртый диск Гражданской Обороны. Конечно, можно было бы съесть всё это и всё это LLM и допилить предметы, какие-то квесты... Но, мне кажется, смысл понятен. А я увлёкся созданием уже БЕСШОВНОГО открытого мира на Godot.

Тестируем и пишем мнения в комментариях к этой статье: <https://ruinsworld.online/>

Теги: `three.js`, `typescript`, `nestjs`, `vue`, `websockets`, `gamedev`

Хабы: Браузеры, Веб-разработка, TypeScript, Дизайн игр, Киберпанк

Электронная почта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



12

Карма

Левон Алексеевич Гамбарян @ushliypakostnik

Создатель игры Ruinsworld и библиотеки UUI

Подписаться



 Комментировать

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



GlinkinIvan

22 часа назад

Хакер в магазине: изучаем поверхность атаки типичного супермаркета



9 мин



14K



+56



46



39



medicalofficer1111
22 часа назад

Пить коллаген — это развод?



7 мин



9.3K



+37



37



22



arturdumchev
23 часа назад

Калвиаутра путает калвиши: один байт изменил всё



Средний



8 мин



11K

Тutorial



+34



7



7



PatientZero
3 часа назад

Теория мёртвой экономики



21 мин



6.3K

Перевод



+24



14



20



nastyakopi
21 час назад

ИИ-роутер, open-source RAG-платформа, ClickHouse-as-a-service и другие апдейты в продуктах Selectel в июне



7 мин



11K

Дайджест



+23



6



3



inetstar
20 часов назад

Полный геном за \$200 и его анализ в домашних условиях: Just-DNA-Lite, ИИ и пересборка генома. Часть 3



Средний



29 мин



10K

Tutorial

 +20

 17

 4



SLY_G

19 часов назад

Сколько элементарных частиц существует на самом деле?



Простой



9 мин



10K

Перевод

 +19

 16

 11



xraizor

3 часа назад

Усталость от напарника-машины. Изнанка работы с ИИ-код-агентами



Средний



11 мин



5.2K

Мнение

 +16

 7

 8



AWE64

15 часов назад

Мне надоело писать один и тот же код. Поэтому я сделал Featuregen



Средний



6 мин



13K

Кейс

 +16

 14

 4



inspired_xy

17 часов назад

Как мы съездили в Японию и запустили Пополаму — сервис, который знает кто кому должен



Простой



4 мин



11K

Кейс

Из песочницы

 +16

 13

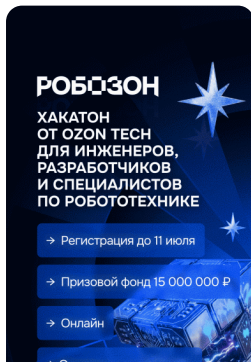
 4

Меряем глубину проникновения ИИ в разработку

Турбо

Показать еще

ИСТОРИИ



Унеси с Робозона свои миллионы



Пришёл, смастерил, рассказал — сезон DIY в разгаре



От лунного овоща до сверхострого чили



Из центра Млечного Пути



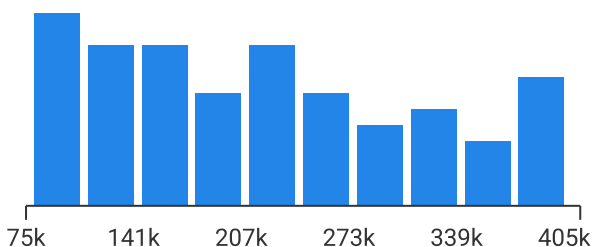
Только IT-пром, только хардкор



СРЕДНЯЯ ЗАРПЛАТА В IT

216 932 ₽/мес.

— средняя зарплата во всех IT-специализациях по данным из 3 464 анкет, за 2-ое пол. 2026 года. Проверьте «в рынке» ли ваша зарплата или нет!



Проверить свою зарплату

МИНУТОЧКУ ВНИМАНИЯ

Турбо

Просто добавь IT: как совмещать код и свёрла в сезоне DIY

Турбо

Моделируем и роботизируем на хакатоне от Ozon Tech

Турбо

Кто твой супергерой в IT? Исследуем работодателей

БЛИЖАЙШИЕ СОБЫТИЯ



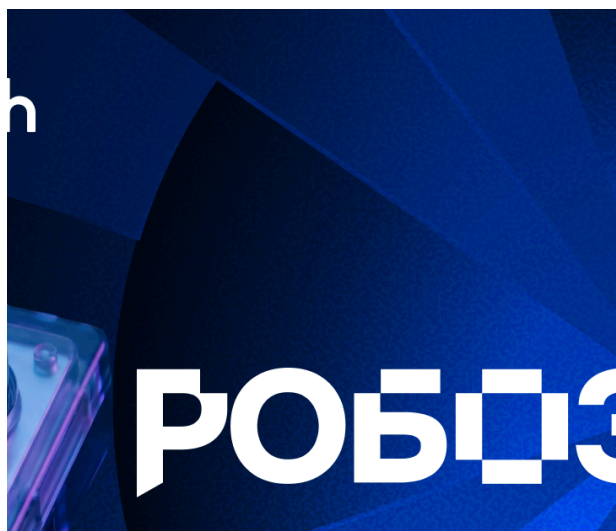
2 марта – 10 августа

PROBA Awards — международная коммуникационная премия, учрежденная в 2000 году

Санкт-Петербург

Маркетинг

Больше событий в календаре



11 июня – 2 сентября

Робозон: хакатон с призовым фондом 15 млн руб.

Москва • Онлайн

Разработка

Больше событий в календаре



15 июля

Веби эпох

Онлайн

Марк

Больше



 [Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr